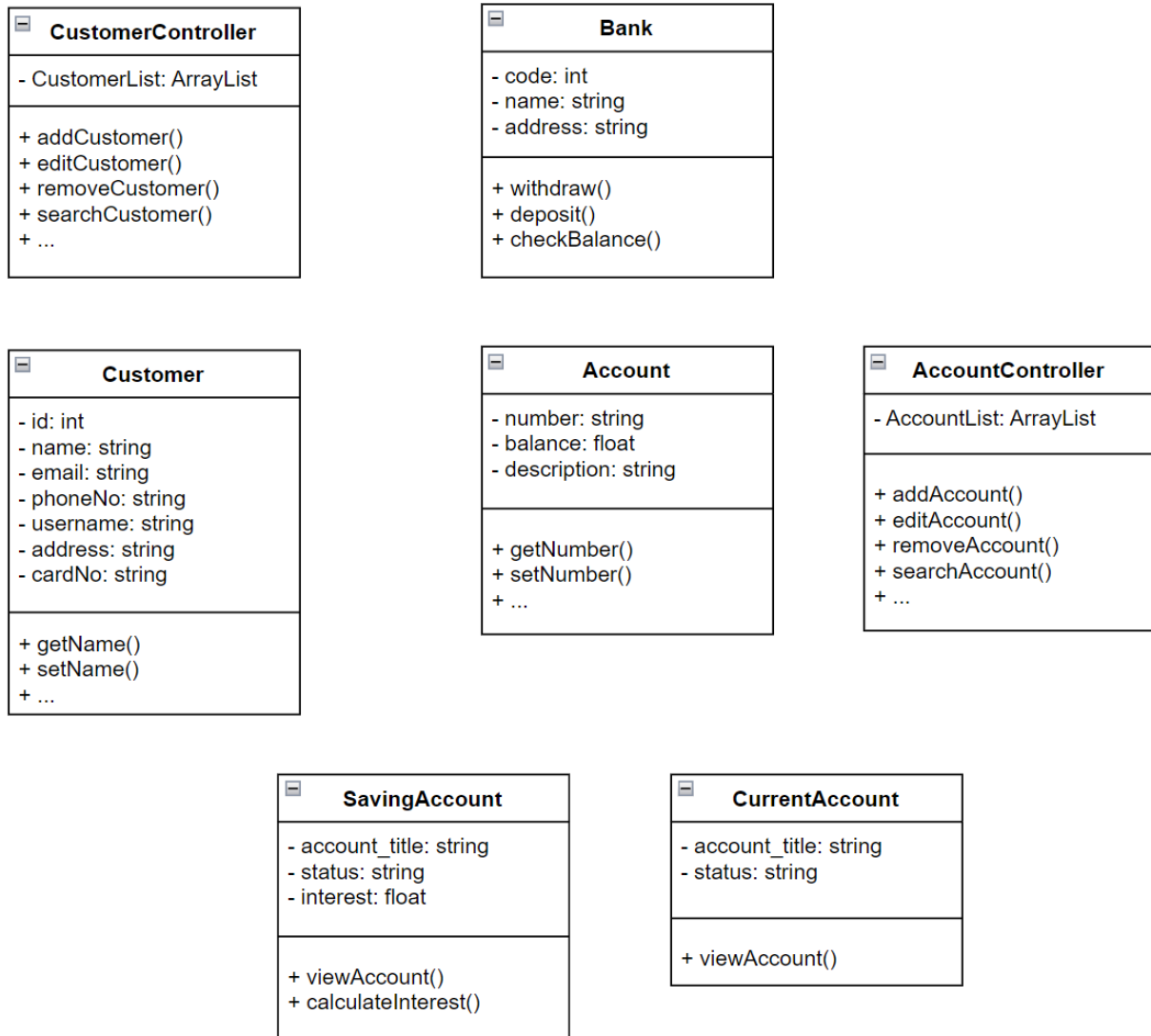


Mini-Project Description

Object Oriented Programming with JAVA

MINI-PROJECT 1

An example of **Bank Management System** is designed including classes as follows:



Requirements:

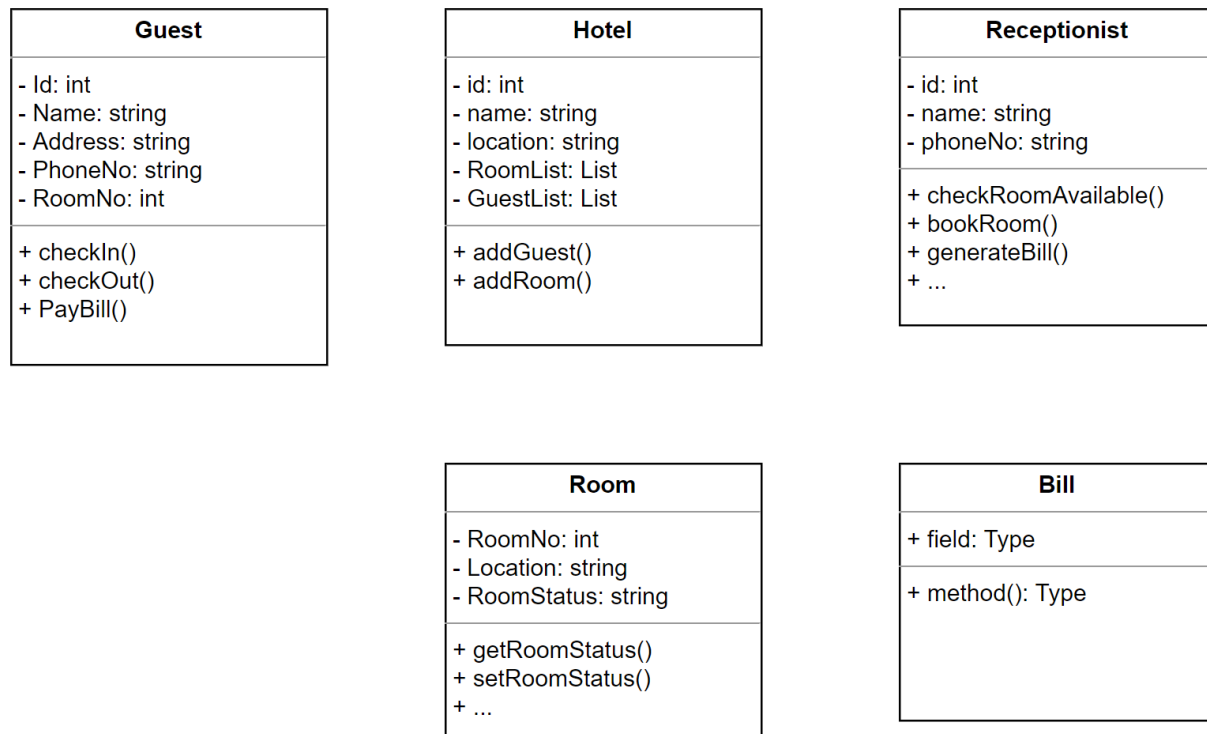
- From the design example above, build a **class diagram** for the system. You can modify or add more fields or methods for your design purpose.
 - Use a UML tool for drawing the class diagram (example draw.io)
 - Explain the class relationships in the diagram
- Applying OOP skills (with Java), build a program that implements the above design with

some features as follows:

- a. Write classes with fields and methods according to the design, use get, set method for the fields.
- b. Use appropriate data structures (example ArrayList) to store the list of objects that are needed to manage such as Account, Customer. Provide basic operation including add/edit/remove the object in the list. These functions should be use by a menu selection in console (or GUI if possible).
- c. Write a Demo class to implement a scenario with the following functions:
 - open an account for a customer, add the account to the list
 - search the account from the list
 - deposit an amount of money for an account
 - withdraw an amount of money for an account
 - check (display) the balance of an account

MINI-PROJECT 2

An example of **Hotel Management System** is designed including classes as follows:



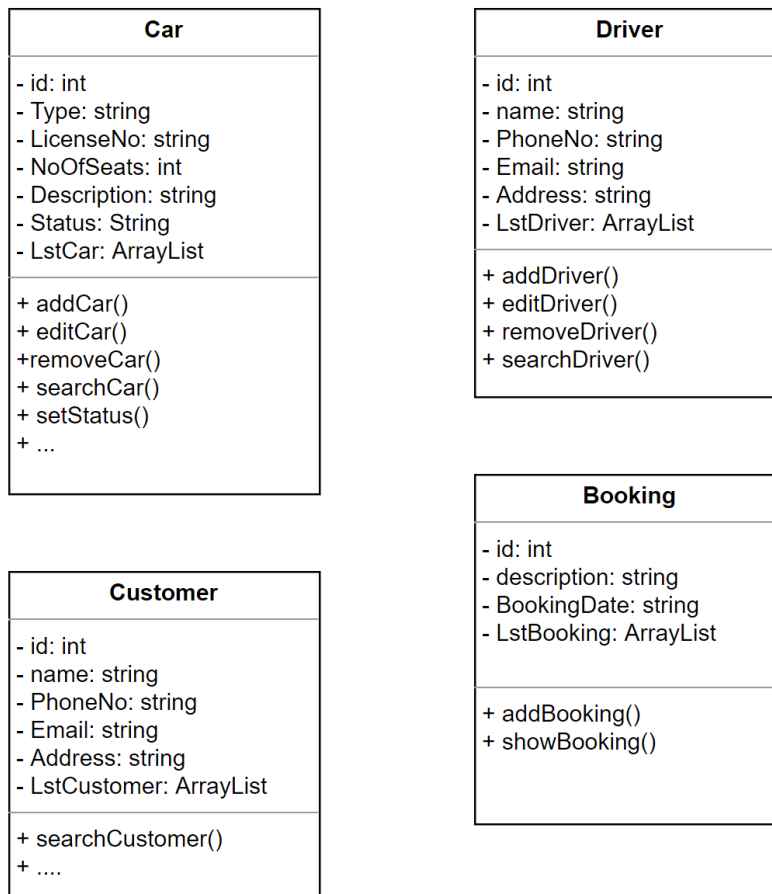
Requirements:

1. From the design example above, build a **class diagram** for the system. You can modify or add more fields or methods for your design purpose.
 - a. Use a UML tool for drawing the class diagram (example draw.io)
 - b. Explain the class relationships in the diagram

2. Applying OOP skills (with Java), build a program that implements the above design with some features as follows:
 - a. Write classes with fields and methods according to the design, use get, set method for the fields.
 - b. Use appropriate data structures (example ArrayList) to store the list of objects that are needed to manage such as Guest, Room. Provide basic operation including add/edit/remove the object in the list. These functions should be use by a menu selection in console (or GUI if possible).
 - c. Write a Demo class to implement a scenario with the following functions:
 - Create a new guest
 - Search an available room for the guest from the list. The room should have a value of room status (empty/occupy)
 - Do the check-in task by showing the information of the guest and the assigned room
 - Do the check-out task by showing the information and change the room status
 - Do the payment task by showing the information

MINI-PROJECT 3

An example of **Car Booking Management System** is designed including classes as follows:



Requirements:

1. From the design example above, build a **class diagram** for the system. You can modify or add more fields or methods for your design purpose.
 - a. Use a UML tool for drawing the class diagram (example draw.io)
 - b. Explain the class relationships in the diagram

2. Applying OOP skills (with Java), build a program that implements the above design with some features as follows:
 - a. Write classes with fields and methods according to the design, use get, set method for the fields.
 - b. Use appropriate data structures (example ArrayList) to store the list of objects that are needed to manage such as Car, Driver, Customer. Provide basic operation including add/edit/remove the object in the list. These functions should be use by a menu selection in console (or GUI if possible).
 - c. Write a Demo class to implement a scenario with the following functions:
 - Create a car booking for a customer
 - Search a car (with free status) and a driver (with top rating point) from the lists of objects for the booking
 - Save the booking to a booking list and show the booking information

MINI-PROJECT 4

An example of **Book Shop Management System** is designed including classes as follows:

Customer
- CustomerID: int - Name: string - PhoneNo: String - UserName: String - Password: String - ShippingAddress: String - BillingAddress: String - LstCustomer: ArrayList
+ addCustomer() + editCustomer() + searchCustomer() + ...

Book
- BookID: int - Title: string - Author: String - Type: String - Publisher: String - PublishYear: date - Price: float - LstBook: ArrayList
+ addBook() + editBook() + removeBook() + searchBook() + ...

ShoppingCart
- CartID: int - customer: Customer - shippingFee: float - Total: float - PaymentMethod: String
+ addItem() + displayCart() + saveCart() + ...

CartItem
- book: Book - quantity: int - price: float
+ addItem() + editItem() + removeItem()

Requirements:

- From the design example above, build a **class diagram** for the system. You can modify or add more fields or methods for your design purpose.
 - Use a UML tool for drawing the class diagram (example draw.io)
 - Explain the class relationships in the diagram
- Applying OOP skills (with Java), build a program that implements the above design with some features as follows:
 - Write classes with fields and methods according to the design, use get, set method for the fields.
 - Use appropriate data structures (example ArrayList) to store the list of objects that are needed to manage such as Book, Customer, Cart. Provide basic operation including add/edit/remove the object in the list. These functions should be use by a menu selection in console (or GUI if possible).
 - Write a Demo class to implement a scenario with the following functions:
 - Create a shopping cart for a customer
 - Search and add book items to the cart
 - Do the Payment Task by showing the information and save to the list

Note:

- Using console program with a menu for selection of functions, data input from keyboard
- If possible, use a GUI for your program, for example: web view, window view
- If possible, use files (txt, csv) or database for management of data in your program.

Submission:

- A short report (.doc file) for mini-project as requirements
- Source code of mini-project